

36 LinkedList Merge Sort For Linked lists.[Very Important]

```
class Solution {
private:
    // Find middle using slow-fast pointers
    Node* getMid(Node* head) {
        Node* slow = head;
        Node* fast = head->next;
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }
    // Merge two sorted lists
    Node* merge(Node* a, Node* b) {
        if (!a) return b;
        if (!b) return a;

        Node* head = NULL;
        if (a->data <= b->data) {
            head = a;
            head->next = merge(a->next, b);
        } else {
            head = b;
            head->next = merge(a, b->next);
        }
        return head;
    }
public:
    Node* mergeSort(Node* head) {
        if (!head || !head->next) return head;
        Node* mid = getMid(head);
        Node* second = mid->next;
        mid->next = NULL; // split list into two halves
        Node* left = mergeSort(head);
        Node* right = mergeSort(second);
        return merge(left, right);
    }
};
```

The screenshot displays a coding platform interface. On the left, the 'Output Window' shows 'Compilation Results' for a submission by 'Y.O.G.I. (AI Bot)'. It indicates 'Problem Solved Successfully' with 11/11 test cases passed, 4/4 points scored, and a total score of 130. On the right, the code editor shows the C++ implementation of the merge sort algorithm for linked lists, which matches the code provided in the text above. The code includes functions for finding the middle node, merging two sorted lists, and the main merge sort function.